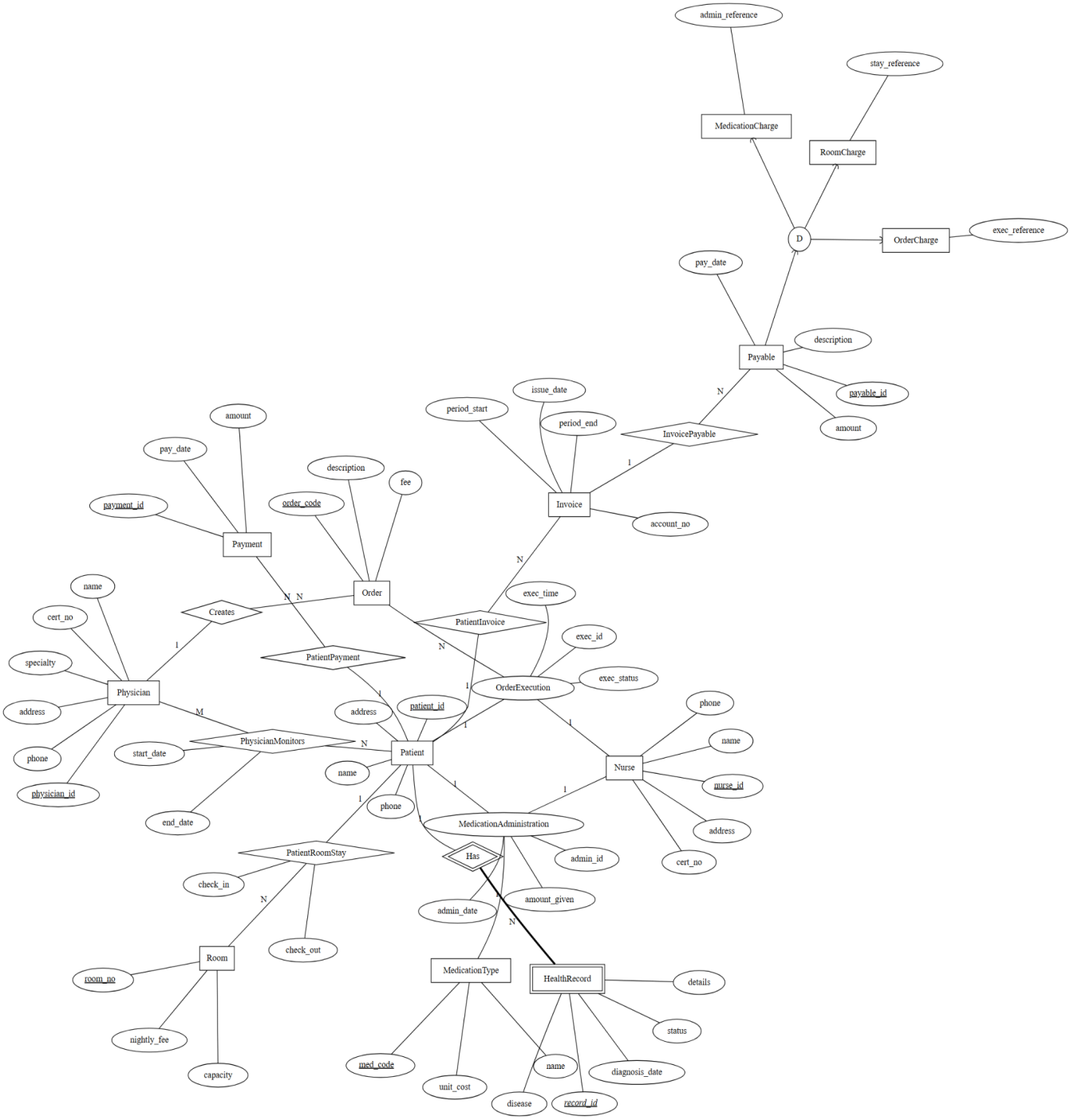
Hospital Management Database System

Mithil Ravulapalli

(E)ER Design:



Assumptions

- A physician can supervise multiple nurses, but each nurse is supervised by only one physician at a time.
- A patient can be assigned to only one room at any given time.
- Medication types are pre-defined in a lookup table to ensure consistency.

Relational Mapping:

Strong Entities:

Patient(patient_id, name, address, phone) primary key: {patient_id} foreign key: {}

Room(room_no, capacity, nightly_fee) primary key: {room_no} foreign key: {}

Physician(physician_id, name, cert_no, specialty, address, phone) primary key: {physician_id} foreign key: {}

Nurse(nurse_id, name, cert_no, address, phone) primary key: {nurse_id} foreign key: {}

Order(order_code, description, fee) primary key: {order_code} foreign key: {}

MedicationType(med_code, name, unit_cost) primary key: {med_code} foreign key: {}

Payment(payment_id, pay_date, amount, patient_id) primary key: {payment_id} foreign key: {patient_id references Patient(patient_id)}

Weak Entities:

HealthRecord(record_id, patient_id, disease, diagnosis_date, status, details) primary key: {record_id, patient_id} foreign key: {patient_id references Patient(patient_id)}

Relationships as Tables:

PatientRoomStay(stay_id, patient_id, room_no, check_in, check_out) primary key: {stay_id} foreign key: {patient_id references Patient(patient_id), room_no references Room(room_no)}

PhysicianMonitors(monitor_id, physician_id, patient_id, start_date, end_date) primary key: {monitor_id} foreign key: {physician_id references Physician(physician_id), patient_id references Patient(patient_id)}

OrderExecution(exec_id, nurse_id, order_code, patient_id, exec_status, exec_time) primary key: {exec_id} foreign key: {nurse_id references Nurse(nurse_id), order_code references Order(order_code), patient_id references Patient(patient_id)}

MedicationAdministration(admin_id, nurse_id, patient_id, med_code, admin_date, amount_given) primary key: {admin_id} foreign key: {nurse_id references Nurse(nurse_id), patient_id references Patient(patient_id), med_code references MedicationType(med_code)}

Creates(physician_id, order_code, creation_date) primary key: {physician_id, order_code} foreign key: {physician_id references Physician(physician_id), order_code references Order(order_code)}

Invoice and Payable System:

Invoice(account_no, issue_date, patient_id, period_start, period_end) primary key: {account_no, issue_date} foreign key: {patient_id references Patient(patient_id)}

Payable(payable_id, patient_id, amount, pay_date, description, payable_type) primary key: {payable_id} foreign key: {patient_id references Patient(patient_id)}

RoomCharge(payable_id, stay_id) primary key: {payable_id} foreign key: {payable_id references Payable(payable_id), stay_id references PatientRoomStay(stay_id)}

MedicationCharge(payable_id, admin_id) primary key: {payable_id} foreign key: {payable_id references Payable(payable_id), admin_id references MedicationAdministration(admin_id)}

OrderCharge(payable_id, exec_id) primary key: {payable_id} foreign key: {payable_id references Payable(payable_id), exec_id references OrderExecution(exec_id)}

InvoicePayable(account_no, issue_date, payable_id) primary key: {account_no, issue_date, payable_id} foreign key: {account_no, issue_date references Invoice(account_no, issue_date), payable_id references Payable(payable_id)}

Queries:

Category A: Join Queries

Query 1: Currently Admitted Patients with Care Team

Description: This query retrieves information about all currently admitted patients along with their assigned rooms, monitoring physicians, and attending nurses.

SQL Query:

```
SELECT DISTINCT p.name AS patient_name,
    r.room_no,
    ph.name AS physician_name,
    n.name AS nurse_name
FROM Patient p
JOIN PatientRoomStay prs ON p.patient_id = prs.patient_id
JOIN Room r ON prs.room_no = r.room_no
JOIN PhysicianMonitors pm ON p.patient_id = pm.patient_id
JOIN Physician ph ON pm.physician_id = ph.physician_id
JOIN OrderExecution oe ON p.patient_id = oe.patient_id
JOIN Nurse n ON oe.nurse_id = n.nurse_id

WHERE prs.check_out IS NULL;
```

	patient_name	room_no	physician_name	nurse_name
▶	Lisa Anderson	201	Dr. Martha Stewart	Nurse Kevin Rodriguez

Query 2: Patient Invoice Details

Description: This query provides detailed invoice information for all patients, showing their payable items with descriptions, amounts, and types.

SQL Query:

```
SELECT p.name AS patient_name,
    i.account_no,
    i.issue_date AS invoice_date,
    py.description AS payable_description,
    py.amount AS payable_amount,
    py.pay_date AS payable_date,
    py.payable_type
FROM Patient p
JOIN Invoice i ON p.patient_id = i.patient_id
JOIN InvoicePayable ip ON i.account_no = ip.account_no AND i.issue_date = ip.issue_date
JOIN Payable py ON ip.payable_id = py.payable_id

ORDER BY p.name, i.issue_date, py.pay_date;
```

patient_name	account_no	invoice_date	payable_description	payable_amount	payable_date	payable_type
David Wilson	ACC005	2024-07-25	Room charges for 5 nights	1500.00	2024-07-25	room
Emily Davis	ACC004	2024-07-12	Room charges for 2 nights	360.00	2024-07-12	room
John Smith	ACC001	2024-07-05	Blood pressure monitoring	45.00	2024-07-01	order
John Smith	ACC001	2024-07-05	ECG monitoring	75.00	2024-07-01	order
John Smith	ACC001	2024-07-05	Lisinopril medication	15.25	2024-07-01	medication
John Smith	ACC001	2024-07-05	Room charges for 4 nights	1000.00	2024-07-05	room
Michael Brown	ACC003	2024-06-20	Room charges for 5 nights	900.00	2024-06-20	room
Sarah Johnson	ACC002	2024-07-07	Chest X-ray	150.00	2024-07-02	order
Sarah Johnson	ACC002	2024-07-07	Amoxicillin medication	25.00	2024-07-02	medication
Sarah Johnson	ACC002	2024-07-07	Room charges for 5 nights	1250.00	2024-07-07	room

Query 3: Physician Patient Monitoring Status

Description: This query shows which physicians are monitoring which patients, including monitoring duration and status.

SQL Query:

```
SELECT ph.name AS physician_name,
    ph.specialty,
    p.name AS patient_name,
    pm.start_date,
    pm.end_date,
    CASE
        WHEN pm.end_date IS NULL THEN 'Currently Monitoring'
        ELSE 'Monitoring Completed'
    END AS monitoring_status
FROM Physician ph
JOIN PhysicianMonitors pm ON ph.physician_id = pm.physician_id
JOIN Patient p ON pm.patient_id = p.patient_id

ORDER BY ph.name, pm.start_date;
```

physician_name	specialty	patient_name	start_date	end_date	monitoring_status
Dr. Allison Cameron	Emergency Medicine	Emily Davis	2024-07-10	2024-07-12	Monitoring Completed
Dr. Eric Foreman	Radiology	David Wilson	2024-07-20	2024-07-25	Monitoring Completed
Dr. James Wilson	Cardiology	John Smith	2024-07-01	2024-07-05	Monitoring Completed
Dr. James Wilson	Cardiology	Robert Taylor	2024-07-22	NULL	Currently Monitoring
Dr. Lisa House	Neurology	Sarah Johnson	2024-07-02	2024-07-07	Monitoring Completed
Dr. Martha Stewart	Pediatrics	Lisa Anderson	2024-07-15	NULL	Currently Monitoring
Dr. Robert Chase	Internal Medicine	Michael Brown	2024-06-15	2024-06-20	Monitoring Completed

Category B: Aggregation Queries

Query 4: Monthly Revenue Analysis

Description: This query analyzes hospital revenue by month and year, showing total revenue and number of charges.

SQL Query:

```
SELECT YEAR(pay_date) AS year,
    MONTH(pay_date) AS month,
    SUM(amount) AS total_revenue,
    COUNT(*) AS number_of_charges
FROM Payable
GROUP BY YEAR(pay_date), MONTH(pay_date)
ORDER BY year DESC, month DESC;
```

year	month	total_revenue	number_of_charges
2024	7	4420.25	9
2024	6	900.00	1

Query 5: Physician Workload Summary

Description: This query summarizes physician workload by counting patients monitored and orders created.

SQL Query:

```
SELECT ph.name AS physician_name,
    ph.specialty,
    COUNT(DISTINCT pm.patient_id) AS patients_monitored,
    COUNT(DISTINCT c.order_code) AS orders_created
FROM Physician ph
LEFT JOIN PhysicianMonitors pm ON ph.physician_id = pm.physician_id
LEFT JOIN Creates c ON ph.physician_id = c.physician_id
GROUP BY ph.physician_id, ph.name, ph.specialty
ORDER BY patients_monitored DESC, orders_created DESC;
```

physician_name	specialty	patients_monitored	orders_created
Dr. James Wilson	Cardiology	2	2
Dr. Lisa House	Neurology	1	1
Dr. Robert Chase	Internal Medicine	1	1
Dr. Allison Cameron	Emergency Medicine	1	1
Dr. Eric Foreman	Radiology	1	1
Dr. Martha Stewart	Pediatrics	1	1

Query 6: Patient Medication Usage Statistics

Description: This query provides statistics on medication administration for each patient.

SQL Query:

```
SELECT p.name AS patient_name,
    COUNT(ma.admin_id) AS medication_administrations,
    AVG(ma.amount_given) AS average_medication_amount,
    MAX(ma.amount_given) AS maximum_medication_amount,
    SUM(ma.amount_given) AS total_medication_given
FROM Patient p
JOIN MedicationAdministration ma ON p.patient_id = ma.patient_id
GROUP BY p.patient_id, p.name
ORDER BY total_medication_given DESC;
```

patient_name	medication_administrations	average_medication_amount	maximum_medication_amount	total_medication_given
David Wilson	1	3.000000	3.00	3.00
John Smith	2	1.000000	1.00	2.00
Sarah Johnson	1	2.000000	2.00	2.00
Emily Davis	1	1.500000	1.50	1.50
Michael Brown	1	1.000000	1.00	1.00
Lisa Anderson	1	1.000000	1.00	1.00

Category C: Nested/Correlated Queries

Query 7: Patients with Outstanding Balances

Description: This nested query identifies patients who have outstanding balances by comparing total charges with total payments.

SQL Query:

```
SELECT p.patient_id, p.name,
    (SELECT COALESCE(SUM(amount), 0) FROM Payable WHERE patient_id =
p.patient_id) AS total_charges,
    (SELECT COALESCE(SUM(amount), 0) FROM Payment WHERE patient_id =
p.patient_id) AS total_payments,
    (SELECT COALESCE(SUM(amount), 0) FROM Payable WHERE patient_id =
p.patient_id) -
    (SELECT COALESCE(SUM(amount), 0) FROM Payment WHERE patient_id =
p.patient_id) AS outstanding_balance
FROM Patient p
```

```
WHERE (SELECT COALESCE(SUM(amount), 0) FROM Payable WHERE patient_id =
p.patient_id) >
    (SELECT COALESCE(SUM(amount), 0) FROM Payment WHERE patient_id =
p.patient_id);
```

patient_id	name	total_charges	total_payments	outstanding_balance
4	Emily Davis	360.00	200.00	160.00

Query 8: Physicians with Above-Average Order Fees

Description: This query finds physicians whose average order fees are above the hospital average.

SQL Query:

```
SELECT ph.name AS physician_name,
    COUNT(c.order_code) AS orders_created,
    AVG(o.fee) AS avg_order_fee
FROM Physician ph
JOIN Creates c ON ph.physician_id = c.physician_id
JOIN `Order` o ON c.order_code = o.order_code
GROUP BY ph.physician_id, ph.name
HAVING AVG(o.fee) > (SELECT AVG(fee) FROM `Order`);
```

physician_name	orders_created	avg_order_fee
Dr. Lisa House	1	150.000000
Dr. Robert Chase	1	85.000000
Dr. Eric Foreman	1	150.000000
Dr. Martha Stewart	1	120.000000

Query 9: Rooms with Below-Average Occupancy

Description: This query identifies rooms with occupancy below the hospital average.

SQL Query:

```
SELECT r.room_no,
    r.capacity,
    COUNT(prs.patient_id) AS current_occupancy,
    (r.capacity - COUNT(prs.patient_id)) AS available_spots
```

```
FROM Room r
LEFT JOIN PatientRoomStay prs ON r.room_no = prs.room_no AND prs.check_out IS NULL
GROUP BY r.room_no, r.capacity
HAVING COUNT(prs.patient_id) < (
   SELECT AVG(occupancy_count)
   FROM (
      SELECT COUNT(patient_id) AS occupancy_count
      FROM PatientRoomStay
      WHERE check_out IS NULL
      GROUP BY room_no
   ) AS avg_occupancy
)
ORDER BY available_spots DESC;
```

room_no	capacity	current_occupancy	available_spots
103	2	0	2
104	2	0	2
101	1	0	1
102	1	0	1
105	1	0	1

Category D: Advanced Queries

Query 10: Patient Stay Duration and Costs

Description: This query calculates stay duration and associated costs for all patient room stays.

SQL Query:

```
SELECT p.name AS patient_name,
      prs.check_in,
      prs.check_out,
      DATEDIFF(COALESCE(prs.check_out, CURDATE()), prs.check_in) AS
stay_duration_days,
      r.nightly_fee,
      (DATEDIFF(COALESCE(prs.check_out, CURDATE()), prs.check_in) * r.nightly_fee) AS
total_room_cost,
      CASE
         WHEN prs.check_out IS NULL THEN 'Currently Admitted'
         ELSE 'Discharged'
      END AS status
```

```
FROM Patient p
JOIN PatientRoomStay prs ON p.patient_id = prs.patient_id
JOIN Room r ON prs.room_no = r.room_no

ORDER BY stay_duration_days DESC;
```

patient_name	check_in	check_out	stay_duration_days	nightly_fee	total_room_cost	status
Lisa Anderson	2024-07-15	NULL	381	275.00	104775.00	Currently Admitted
Robert Taylor	2024-07-22	NULL	374	200.00	74800.00	Currently Admitted
Sarah Johnson	2024-07-02	2024-07-07	5	250.00	1250.00	Discharged
Michael Brown	2024-06-15	2024-06-20	5	180.00	900.00	Discharged
David Wilson	2024-07-20	2024-07-25	5	300.00	1500.00	Discharged
John Smith	2024-07-01	2024-07-05	4	250.00	1000.00	Discharged
Emily Davis	2024-07-10	2024-07-12	2	180.00	360.00	Discharged

Query 11: Patients with Multiple Health Conditions

Description: This query identifies patients with multiple health conditions and lists their conditions.

SQL Query:

```
SELECT p.patient_id,
    p.name AS patient_name,
    COUNT(hr.record_id) AS number_of_conditions,
    GROUP_CONCAT(hr.disease SEPARATOR ', ') AS conditions_list
FROM Patient p
JOIN HealthRecord hr ON p.patient_id = hr.patient_id
GROUP BY p.patient_id, p.name
HAVING COUNT(hr.record_id) > 1

ORDER BY number_of_conditions DESC;
```

patient_id	patient_name	number_of_conditions	conditions_list
1	John Smith	2	Hypertension, Type 2 Diabetes

Query 12: Physicians with No Current Patients

Description: This query identifies physicians who are not currently monitoring any patients.

SQL Query:

```
SELECT n.name AS nurse_name,
```

```
    COUNT(DISTINCT oe.exec_id) AS orders_executed,
    COUNT(DISTINCT ma.admin_id) AS medications_given,
    COUNT(DISTINCT oe.patient_id) AS patients_served,
    ROUND((COUNT(DISTINCT oe.exec_id) * 100.0 /
        (SELECT COUNT(*) FROM OrderExecution)), 2) AS percentage_of_total_orders
FROM Nurse n
LEFT JOIN OrderExecution oe ON n.nurse_id = oe.nurse_id
LEFT JOIN MedicationAdministration ma ON n.nurse_id = ma.nurse_id
GROUP BY n.nurse_id, n.name
ORDER BY orders_executed DESC, medications_given DESC;
```

nurse_name	orders_executed	medications_given	patients_served	percentage_of_total_orders
Nurse Jennifer Adams	2	2	1	28.57
Nurse Michael Clark	1	1	1	14.29
Nurse Susan Miller	1	1	1	14.29
Nurse Daniel Garcia	1	1	1	14.29
Nurse Patricia Martinez	1	1	1	14.29
Nurse Kevin Rodriguez	1	1	1	14.29

Query 13: Monthly Payment Analysis

Description: This query analyzes payment patterns by month and year.

SQL Query:

```
SELECT YEAR(pay_date) AS payment_year,
    MONTH(pay_date) AS payment_month,
    COUNT(*) AS number_of_payments,
    SUM(amount) AS total_payments_received,
    AVG(amount) AS average_payment_amount
FROM Payment
GROUP BY YEAR(pay_date), MONTH(pay_date)
ORDER BY payment_year DESC, payment_month DESC;
```

payment_year	payment_month	number_of_payments	total_payments_received	average_payment_amount
2024	7	6	4260.25	710.041667
2024	6	1	900.00	900.000000

Query 14: Patients Without Recent Invoices

Description: This query identifies patients who have had room stays but no recent invoices.

SQL Query:

```
SELECT p.patient_id,
      p.name AS patient_name,
      p.phone,
      COUNT(prs.stay_id) AS number_of_stays
FROM Patient p
JOIN PatientRoomStay prs ON p.patient_id = prs.patient_id
WHERE NOT EXISTS (
   SELECT 1
   FROM Invoice i
   WHERE i.patient_id = p.patient_id
   AND i.issue_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
)
GROUP BY p.patient_id, p.name, p.phone
ORDER BY number_of_stays DESC;
```

patient_id	patient_name	phone	number_of_stays
1	John Smith	312-555-0101	1
2	Sarah Johnson	312-555-0102	1
3	Michael Brown	312-555-0103	1
4	Emily Davis	312-555-0104	1
5	David Wilson	312-555-0105	1
6	Lisa Anderson	312-555-0106	1
7	Robert Taylor	312-555-0107	1

Query 15: Top Revenue-Generating Orders

Description: This query identifies the highest revenue-generating order types.

SQL Query:

```
SELECT o.order_code,
```

```
    o.description,
    o.fee,
    COUNT(oe.exec_id) AS times_executed,
    SUM(o.fee) AS total_revenue_generated
FROM `Order` o
LEFT JOIN OrderExecution oe ON o.order_code = oe.order_code
GROUP BY o.order_code, o.description, o.fee
ORDER BY o.fee DESC
LIMIT 5;
```

order_code	description	fee	times_executed	total_revenue_generated
ORD005	X-ray chest	150.00	2	300.00
ORD004	Physical therapy session	120.00	1	120.00
ORD002	IV fluid administration	85.00	1	85.00
ORD006	ECG monitoring	75.00	1	75.00
ORD003	Wound dressing change	65.00	1	65.00

Views:

View 1: v_Patient_Balance

Description: This view provides a comprehensive overview of each patient's financial status by calculating their total charges, payments, and current balance. It automatically categorizes patients as having outstanding balances, being overpaid, or having settled accounts.

SQL Body:

```
CREATE VIEW v_Patient_Balance AS
SELECT p.patient_id,
    p.name AS patient_name,
    p.address,
    p.phone,
    COALESCE(SUM(pay.amount), 0) AS total_charges,
    COALESCE(SUM(pmt.amount), 0) AS total_payments,
    COALESCE(SUM(pay.amount), 0) - COALESCE(SUM(pmt.amount), 0) AS
current_balance,
    CASE
        WHEN COALESCE(SUM(pay.amount), 0) - COALESCE(SUM(pmt.amount), 0) > 0
THEN 'Outstanding'
```

```
        WHEN COALESCE(SUM(pay.amount), 0) - COALESCE(SUM(pmt.amount), 0) < 0
THEN 'Overpaid'
        ELSE 'Settled'
    END AS balance_status
FROM Patient p
LEFT JOIN Payable pay ON p.patient_id = pay.patient_id
LEFT JOIN Payment pmt ON p.patient_id = pmt.patient_id

GROUP BY p.patient_id, p.name, p.address, p.phone;
```

Usefulness: This view is essential for the hospital's billing department as it provides immediate access to patient financial status without requiring complex joins across multiple tables. It enables quick identification of patients with outstanding balances for follow-up and helps maintain accurate financial records.

View 2: v_Medication_Summary

Description: This view provides comprehensive statistics about medication usage, including administration frequency, total costs, and patient coverage.

SQL Body:

```
CREATE VIEW v_Medication_Summary AS
SELECT mt.med_code,
    mt.name AS medication_name,
    mt.unit_cost,
    COUNT(ma.admin_id) AS times_administered,
    SUM(ma.amount_given) AS total_amount_used,
    SUM(ma.amount_given * mt.unit_cost) AS total_medication_cost,
    AVG(ma.amount_given) AS average_dose,
    COUNT(DISTINCT ma.patient_id) AS patients_treated
FROM MedicationType mt
LEFT JOIN MedicationAdministration ma ON mt.med_code = ma.med_code
GROUP BY mt.med_code, mt.name, mt.unit_cost
ORDER BY total_medication_cost DESC;
```

Usefulness: This view is valuable for pharmacy management and cost control, enabling tracking of medication usage patterns, inventory planning, and identification of high-cost medications. It supports clinical decision-making and helps optimize medication procurement strategies.

View 3: v_Physician_Workload

Description: This view summarizes each physician's current workload, including the number of patients they're monitoring, orders created, and average order values.

SQL Body:

```
CREATE VIEW v_Physician_Workload AS
SELECT ph.physician_id,
    ph.name AS physician_name,
    ph.specialty,
    COUNT(DISTINCT pm.patient_id) AS current_patients,
    COUNT(DISTINCT c.order_code) AS orders_created,
    AVG(o.fee) AS avg_order_value,
    ph.phone AS contact_phone
FROM Physician ph
LEFT JOIN PhysicianMonitors pm ON ph.physician_id = pm.physician_id AND pm.end_date
IS NULL
LEFT JOIN Creates c ON ph.physician_id = c.physician_id
LEFT JOIN `Order` o ON c.order_code = o.order_code
GROUP BY ph.physician_id, ph.name, ph.specialty, ph.phone;
```

Usefulness: This view supports resource allocation and workload management by providing insights into physician productivity and patient distribution. It helps administrators balance caseloads and identify physicians who may need additional support or those available for new assignments.

Triggers:

Trigger 1: trg_After_OrderExecution_CreateCharge

Description: This trigger automatically creates billing charges when medical orders are executed by nurses, ensuring that all billable services are properly recorded in the payables system.

SQL Body:

```
DELIMITER //
CREATE TRIGGER trg_After_OrderExecution_CreateCharge
AFTER INSERT ON OrderExecution
FOR EACH ROW
BEGIN
```

```
    DECLARE order_fee DECIMAL(10,2);
    DECLARE new_payable_id INT;

    SELECT fee INTO order_fee
    FROM `Order`
    WHERE order_code = NEW.order_code;

    INSERT INTO Payable(patient_id, amount, pay_date, description, payable_type)
    VALUES(NEW.patient_id,
           order_fee,
           NEW.exec_time,
           CONCAT('Order execution: ', NEW.order_code, ' - ',
                (SELECT description FROM `Order` WHERE order_code = NEW.order_code)),
           'order');

    SET new_payable_id = LAST_INSERT_ID();

    INSERT INTO OrderCharge(payable_id, exec_id)
    VALUES(new_payable_id, NEW.exec_id);

END;
//

DELIMITER ;
```

Usefulness: This trigger ensures billing accuracy and completeness by automatically generating charges for executed orders. It eliminates the possibility of unbilled services and maintains data consistency between clinical operations and financial records, reducing administrative overhead and billing errors.

Trigger 2: trg_After_Checkout_CreateRoomCharge

Description: This trigger automatically calculates and records room charges when patients are discharged, based on their length of stay and room rates.

SQL Body:

```
DELIMITER //
CREATE TRIGGER trg_After_Checkout_CreateRoomCharge
AFTER UPDATE ON PatientRoomStay
FOR EACH ROW
BEGIN
```

```
    DECLARE nights_stayed INT;
    DECLARE room_fee DECIMAL(10,2);
    DECLARE total_room_cost DECIMAL(10,2);
    DECLARE new_payable_id INT;

    IF OLD.check_out IS NULL AND NEW.check_out IS NOT NULL THEN

        SET nights_stayed = DATEDIFF(NEW.check_out, NEW.check_in);

        SELECT nightly_fee INTO room_fee
        FROM Room
        WHERE room_no = NEW.room_no;

        SET total_room_cost = nights_stayed * room_fee;

        INSERT INTO Payable(patient_id, amount, pay_date, description, payable_type)
        VALUES(NEW.patient_id,
            total_room_cost,
            NEW.check_out,
            CONCAT('Room ', NEW.room_no, ' charges for ', nights_stayed, ' nights'),
            'room');

        SET new_payable_id = LAST_INSERT_ID();

        INSERT INTO RoomCharge(payable_id, stay_id)
        VALUES(new_payable_id, NEW.stay_id);

    END IF;
END;
//

DELIMITER ;
```

Usefulness: This trigger automates the room billing process, ensuring accurate calculation of accommodation charges based on actual length of stay. It prevents billing errors and ensures that all room usage is properly charged to patients upon discharge.

Trigger 3: trg_After_MedicationAdmin_CreateCharge

Description: This trigger automatically creates billing entries for medication administration, calculating costs based on the type and amount of medication given.

SQL Body:

```
DELIMITER //
CREATE TRIGGER trg_After_MedicationAdmin_CreateCharge
AFTER INSERT ON MedicationAdministration
FOR EACH ROW
BEGIN
   DECLARE med_unit_cost DECIMAL(10,2);
   DECLARE total_med_cost DECIMAL(10,2);
   DECLARE med_name VARCHAR(100);
   DECLARE new_payable_id INT;

   SELECT unit_cost, name
   INTO med_unit_cost, med_name
   FROM MedicationType
   WHERE med_code = NEW.med_code;

   SET total_med_cost = med_unit_cost * NEW.amount_given;

   INSERT INTO Payable(patient_id, amount, pay_date, description, payable_type)
   VALUES(NEW.patient_id,
        total_med_cost,
        NEW.admin_date,
        CONCAT('Medication: ', med_name, ' (', NEW.amount_given, ' units)'),
        'medication');

   SET new_payable_id = LAST_INSERT_ID();

   INSERT INTO MedicationCharge(payable_id, admin_id)
   VALUES(new_payable_id, NEW.admin_id);

END;
//
DELIMITER ;
```

Usefulness: This trigger ensures accurate medication billing by automatically calculating charges based on actual medication usage. It maintains consistency between clinical care delivery and billing systems, preventing revenue loss from unbilled medications.

Transactions:

Transaction 1: usp_AdmitPatient

Description: This stored procedure handles the complete patient admission process, including patient registration, room assignment, and initial health record creation within a single transaction.

```
DELIMITER //

CREATE PROCEDURE usp_AdmitPatient(

    IN p_name VARCHAR(100),

    IN p_address VARCHAR(200),

    IN p_phone VARCHAR(15),

    IN p_room_no INT

)

BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;


    START TRANSACTION;


    INSERT INTO Patient(name, address, phone_number)

    VALUES(p_name, p_address, p_phone);
```

```
    SET @new_patient_id = LAST_INSERT_ID();

    INSERT INTO PatientRoomStay(patient_id, room_no, check_in)
    VALUES(@new_patient_id, p_room_no, CURDATE());

    INSERT INTO HealthRecord(patient_id, description, diagnosis_date, status)
    VALUES(@new_patient_id, 'Initial admission', CURDATE(), 'Active');

    COMMIT;
END //
DELIMITER ;
```

Usefulness: This transaction ensures data consistency during patient admission by treating the entire admission process as an atomic operation. If any step fails, all changes are rolled back, preventing partial admissions that could lead to data inconsistencies. This is crucial for maintaining accurate patient records and room assignments.

Transaction 2: usp_TransferPatient

Description: This stored procedure manages patient room transfers, including checkout from the current room, billing calculation, and check-in to the new room, all within a single transaction.

SQL Body:

```
DELIMITER //
CREATE PROCEDURE usp_TransferPatient(
    IN p_patient_id INT,
```

```
    IN p_new_room_no INT
)
BEGIN
    DECLARE room_charge DECIMAL(10,2);
    DECLARE nights INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT DATEDIFF(CURDATE(), check_in), r.nightly_fee
    INTO nights, room_charge
    FROM PatientRoomStay prs
    JOIN Room r ON prs.room_no = r.room_no
    WHERE prs.patient_id = p_patient_id AND check_out IS NULL;

    UPDATE PatientRoomStay
    SET check_out = CURDATE()
    WHERE patient_id = p_patient_id AND check_out IS NULL;
```

```
    SAVEPOINT after_checkout;

    INSERT INTO Payable(patient_id, amount, description, pay_date)
    VALUES(p_patient_id, room_charge * nights, 'Room charges', CURDATE());

    INSERT INTO PatientRoomStay(patient_id, room_no, check_in)
    VALUES(p_patient_id, p_new_room_no, CURDATE());

    COMMIT;
END //
DELIMITER ;
```

Usefulness: This transaction ensures that patient transfers are handled atomically, preventing situations where a patient might be checked out of one room but not checked into another, or where billing calculations are incomplete. The use of savepoints allows for partial rollback if needed, making the transaction more robust and reliable for complex operations like room transfers.